

In-Class Exercise Sheet 8

1. Fill in the blanks to compare the different data formats, as discussed in the video material.

	Relational	XML	JSON
Structure			
Schema			
Queries			
Ordering			
Implementation			

Based on Jennifer Widom's online database course.

2. Consider the following two JSON instances¹.

<pre> 1 { 2 "productId": 1, 3 "productName": "A green door", 4 "price": 12.50, 5 "tags": ["home", "green"] 6 }</pre>	<pre> { "productId": 1, "productName": "A green door", "tags": ["green", "home"] "price": 12.50, }</pre>
--	--

Are they structurally equivalent?

- ☐ No, because a tool like `diff` will detect syntactic differences in lines 4 and 5.
- ☐ No, because the order of properties differs.
- ☐ No, because the order of array items differs.
- ☐ Yes, because JSON is an unordered data format.

¹Example originally from <https://json-schema.org/learn/getting-started-step-by-step>.

3. Label the following chart illustrating the data flow in schema validation.

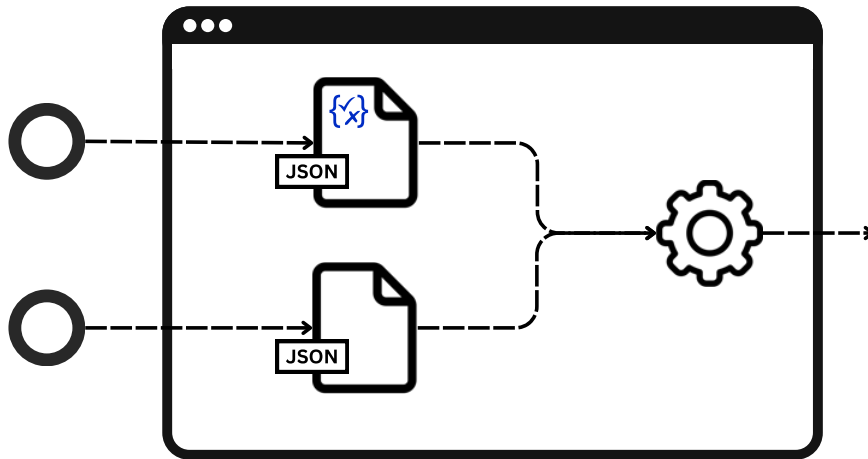


Illustration adapted from the online tutorial from JSON Schema Org at <https://json-schema.org/learn/getting-started-step-by-step>.

4. What are benefits to validating JSON instances against a JSON Schema?

5. Consider the schema shown further below.² Which statements are true?

- ☐ The left JSON instance from question 2 is valid w.r.t. the schema up until line 3.
- ☐ The string "Hello world!" is valid w.r.t. the schema up until (including) line 3.
- ☐ The left JSON instance from question 2 is valid w.r.t. the schema up until (including) line 4.
- ☐ The JSON instance "Hello world!" is valid w.r.t. the schema up until (including) line 4.
- ☐ The JSON instance { "foo": 42 } is valid w.r.t. the schema up until (including) line 28.

²Based on the schema from the online tutorial <https://json-schema.org/learn/getting-started-step-by-step>.

- ☐ The JSON instance { "ProductName": 42 } is valid w.r.t. the schema up until (including) line 28. (⚠)
- ☐ The JSON instance { "ProductName": 42 } is valid w.r.t. the schema. (⚠)
- ☐ The left JSON instance from question 2 is valid w.r.t. the schema.
- ☐ The JSON instance below is valid w.r.t. the schema.

```
{
  "productId": 1,
  "productName": "A green door",
  "price": 12.50,
  "tags": [ ]
}
```

- ☐ The JSON instance below is valid w.r.t. the schema.

```
{
  "productId": 1,
  "productName": "A green door",
  "price": 12.50,
  "tags": [ "special offer", "special offer" ]
}
```

- ☐ The JSON instance below is valid w.r.t. the schema.

```
{
  "productId": 1,
  "productName": "A green door",
  "price": 12.50,
  "tags": [ "home", "green" ],
  "discount": 0.1
}
```

JSON Schema:

```
1 {
2   "title": "Product",
3   "description": "A product from Acme's catalog",
4   "type": "object",
5   "properties": {
6     "productId": {
7       "description": "The unique identifier for a product",
8       "type": "integer"
9     },
10    "productName": {
11      "description": "Name of the product",
12      "type": "string"
13    },
14    "price": {
15      "description": "The price of the product",
16      "type": "number",
17      "exclusiveMinimum": 0
18    },
19    "tags": {
20      "description": "Tags for the product",
21      "type": "array",
22      "items": {
23        "type": "string"
24      },
25      "minItems": 1,
26      "uniqueItems": true
27    }
28  },
29  "required": [ "productId", "productName", "price" ]
30 }
```

6. Consider the JSON Schema below:³

```
1 {
2   "allOf": [
3     { "type": "string" },
4     { "maxLength": 5 }
5   ]
6 }
```

Which of the following JSON instances are valid w.r.t. this schema?

- ☐ "foo"
- ☐ "a"
- ☐ "1234567890"
- ☐ 42

³From: <https://json-schema.org/understanding-json-schema/reference/combining>.

7. Consider the JSON Schema below:

```
1 {  
2   "anyOf": [  
3     { "type": "string" },  
4     { "maxLength": 5 }  
5   ]  
6 }
```

Which of the following JSON instances are valid w.r.t. this schema?

- ☐ "foo"
- ☐ "a"
- ☐ "1234567890"
- ☐ 42

8. Consider the JSON Schema below:

```
1 {  
2   "oneOf": [  
3     { "type": "string" },  
4     { "maxLength": 5 }  
5   ]  
6 }
```

Which of the following JSON instances are valid w.r.t. this schema?

- ☐ "foo"
- ☐ "a"
- ☐ "1234567890"
- ☐ 42

9. Consider the following two JSON Schemas.

```
1 {  
2   "oneOf": [  
3     { "type": "string" },  
4     { "type": "integer" }  
5   ]  
6 }
```

```
{  
  "anyOf": [  
    { "type": "integer" },  
    { "type": "string" }  
  ]  
}
```

Which statements are true?

- ☐ There exists a JSON instance that is valid w.r.t. the right schema but not the left.
- ☐ The schemas are structurally equivalent.
- ☐ The schemas are semantically equivalent.
- ☐ None of these options.

10. When might we prefer the hierarchical HDF5 format over XML or JSON – and when not?

11. Consider wind and temperature data collected from two different weather stations, stored in CSV format.

Temperature						Metadata		
station	time	meas	station	time	meas	station	key	value
15	0	18.2	15	0	3.1	15	delta	5.0
15	5	18.4	15	5	3.3	15	start_time	0
15	10	18.7	15	10	2.8	15	temp_unit	degree Celsius
15	15	19.0	15	15	4.0	15	wind_unit	m/s
15	20	19.1	15	20	3.7	20	delta	10.0
20	0	64.0				20	start_time	0
20	10	65.0				20	temp_unit	degree Fahrenheit
20	20	66.1						
20	30	65.8						

- (a) What are the benefits and drawbacks of the tabular layout above?
- (b) You intend to store the data in HDF5 format, structured as follows:

```

/
+---15
|   +---temperature
|   +---wind
|
+---20
    +---temperature
  
```

Complete the Python code below accordingly.

```
import time
import numpy as np
import h5py

def main():
    # Keeping things very simple here.
    temperature_station_15 = np.array([18.2, 18.4, 18.7, 19.0, 19.1])
    wind_station_15 = np.array([3.1, 3.3, 2.8, 4.0, 3.7])
    temperature_station_20 = np.array([64.0, 65.0, 66.1, 65.8])
    start_time = 0 # Should be a proper timestamp.

    with h5py.File("weather.hdf5", "w") as f:

        # Store data for weather station 15

        f[_____] = temperature_station_15

        f[_____].attrs[_____] = 5.0

        f[_____].attrs[_____] = start_time

        f[_____].attrs[_____] = "degree Celsius"

        f[_____] = wind_station_15

        f[_____].attrs[_____] = 5.0

        f[_____].attrs[_____] = start_time

        f[_____].attrs[_____] = "m/s"

        # Store data for weather station 20

        f[_____] = temperature_station_20

        f[_____].attrs[_____] = 10.0

        f[_____].attrs[_____] = start_time

        f[_____].attrs[_____] = "degree Fahrenheit"

        f.attrs["description"] = _____

if __name__ == "__main__":
    main()
```

Based on the O'Reilly book "Python and HDF5" by Andrew Colette (with small changes).